

Scenario 1: Bad Container Image Deployment

Objective

Trace an incomplete Kubernetes rollout from a Datadog signal to the Deployment image and Amazon ECR, then recover safely.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/bad-deploy.sh banking banking-backend banking-backend
```

The script changes the live Deployment image to a nonexistent tag. Wait for Datadog and Kubernetes to observe the failed rollout before troubleshooting. Do not run another scenario until this one has been recovered.

Situation

The banking application still serves users, but a new release has not completed. Do not run the trigger yourself unless you are facilitating the lab.

Symptoms

- The rollout does not complete.
- New pods do not start while existing pods may remain healthy.

What You Know

- The issue began immediately after a banking backend release.
- Datadog shows the `banking-backend` Deployment with unavailable replicas and new pods that are not Ready.
- Existing pods may still serve traffic.

Start With Datadog

1. Open **Dashboards > SRE Lab Scenario Signals** and set the time range to the last 15 minutes.
2. In **Unavailable Replicas**, filter `kube_namespace:banking` and `kube_deployment:banking-backend`. Look for `kubernetes_state.deployment.replicas_unavailable` rising above zero.

3. Compare **Available Replicas**. Existing capacity may stay healthy while the new rollout is unavailable.
4. Open **Monitors > Manage Monitors > [SRE Lab] Deployment has unavailable replicas**. Confirm the alert group names `banking/banking-backend`.
5. Open **Infrastructure > Kubernetes > Deployments**, filter `kube_namespace:banking` `kube_deployment:banking-backend`, and inspect the Deployment and newest ReplicaSet.
6. Open **Events > Explorer**, use the same namespace/workload tags, and look for `Failed`, `ErrImagePull`, `ImagePullBackOff`, or image-pull messages.

Record the first signal timestamp, unavailable replica count, affected ReplicaSet, and event message before using `kubectl`.

Troubleshooting

First find the incomplete rollout and its new pods.

```
kubectl get deployments -A
kubectl get pods -A
kubectl rollout status deployment/banking-backend -n banking
```

Then inspect the newest pod, its events, and the image requested by the Deployment.

```
kubectl describe pod <pod-name> -n banking
kubectl get events -n banking --sort-by=.lastTimestamp
kubectl get deployment banking-backend -n banking -o yaml
```

Compare the configured tag with ECR.

```
aws ecr describe-images --repository-name sre-lab/banking-backend --region us-east-1
```

Important Clues

Connect the Datadog replica mismatch with `ErrImagePull` or `ImagePullBackOff`, Kubernetes events, and the ECR image list.

Root Cause

Record your conclusion from Datadog, Kubernetes, and ECR evidence. Confirm it with the instructor after the exercise.

Fix

Roll back to the last working revision.

```
kubectl rollout history deployment/banking-backend -n banking
kubectl rollout undo deployment/banking-backend -n banking
```

Validation

```
kubectl rollout status deployment/banking-backend -n banking
kubectl get pods -n banking
```

Confirm the application still works, unavailable replicas return to zero, and the Datadog monitor recovers.

DevOps Lesson

Validate an ECR tag before changing a Kubernetes Deployment. A healthy old ReplicaSet does not mean a release succeeded.

STAR Method

Situation

Summarize the failed rollout without naming the cause first.

Task

Explain the safe recovery objective.

Action

Describe the evidence path from Datadog to Kubernetes and ECR.

Result

State how rollout and observability recovery were verified.

Natural Spoken Version

Use the matching example in [DevOps STAR Scenarios](#) after completing the lab.

Scenario 2: Bad ConfigMap Rollout

Objective

Diagnose a rollout where application configuration and Kubernetes health probes no longer agree.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/break-config.sh student-portal
```

The script saves the current port, changes the ConfigMap, and restarts the student portal backend Deployment. Wait for Datadog and Kubernetes to observe the rollout failure before troubleshooting.

Situation

The student portal has reduced rollout capacity after a configuration release. Existing pods may still serve requests, but new pods do not become Ready.

Symptoms

- New pods fail to become Ready.
- Restart counts may increase while older pods still serve.

What You Know

- Datadog shows unavailable replicas and increasing container restarts for `student-portal-backend`.
- Kubernetes health events began after a rollout.
- The process in each new container appears to start.

Start With Datadog

1. Open **Dashboards > SRE Lab Scenario Signals** with a last-15-minutes window.
2. Filter **Unavailable Replicas** to `kube_namespace:student-portal` and `kube_deployment:student-portal-backend`. Look for a value above zero.
3. In **Container Restarts**, use the same filters and look for a rising `kubernetes.containers.restarts` series.
4. Open **Monitors > Manage Monitors** and inspect **Deployment has unavailable replicas** and **Pod restarts detected**. Expand the `student-portal-backend` alert group.

5. Open **Infrastructure > Kubernetes > Pods** and filter `kube_namespace:student-portal`
`kube_deployment:student-portal-backend` . Compare Ready status and restart counts between older and newer pods.
6. Open **Events > Explorer** with `kube_namespace:student-portal` . Look for readiness/liveness probe failures and BackOff events.
7. Open **Logs > Explorer**, set the last 15 minutes, and search `service:student-portal-backend` . Look for the startup line that reports the listening port.

Record the first unavailable-replica timestamp, newest pod, probe event, restart change, and logged listening port.

Troubleshooting

Check rollout health and identify the newest pods.

```
kubectl get deployments -n student-portal
kubectl get pods -n student-portal
kubectl rollout status deployment/student-portal-backend -n student-portal
kubectl describe pod <pod-name> -n student-portal
kubectl get events -n student-portal --sort-by=.lastTimestamp
kubectl logs <pod-name> -n student-portal
```

After confirming probe failures, compare workload configuration with the probes and rollout history.

```
kubectl get configmaps -n student-portal
kubectl describe configmap student-portal-backend-config -n student-portal
kubectl get deployment student-portal-backend -n student-portal -o yaml
kubectl rollout history deployment/student-portal-backend -n student-portal
```

Important Clues

Connect the probe failures to the application listening port and the port checked by the readiness and liveness probes.

Root Cause

Record the configuration and probe mismatch supported by your evidence. Confirm it with the instructor after the exercise.

Fix

```
./scripts/chaos/break-config.sh student-portal --undo
```

The script restores the saved `PORT` value and restarts the Deployment.

Validation

```
kubectl rollout status deployment/student-portal-backend -n student-portal  
kubectl get pods -n student-portal
```

Confirm every pod is Ready, restarts stop increasing, the app works, and both Datadog monitors recover.

DevOps Lesson

Validate configuration and health-probe contracts together before rollout.

STAR Method

Situation

Summarize the partial-capacity rollout.

Task

Explain why only new pods needed investigation.

Action

Describe the Datadog, event, log, ConfigMap, and probe comparison.

Result

State how readiness and monitor recovery were verified.

Natural Spoken Version

Use the matching example in [DevOps STAR Scenarios](#) after completing the lab.

Scenario 3: OOMKilled and Kubernetes Resource Limits

Objective

Use Datadog and Kubernetes evidence to distinguish memory pressure from an unsafe container resource limit.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/shrink-limits.sh support-tickets
```

The script saves the current resources and applies a deliberately unsafe memory request and limit. Wait for restarts and Datadog signals before troubleshooting.

Situation

The support tickets service is unstable. Backend pods restart repeatedly while some requests still succeed.

Symptoms

- Backend pods restart repeatedly.
- Application stability degrades.

What You Know

- Datadog shows backend memory near its configured ceiling.
- Container restart counts are increasing.
- The problem began after a workload resource change.

Start With Datadog

1. Open **Dashboards > SRE Lab Scenario Signals** and set the time range to the last 15 minutes.
2. In **Memory Usage and Limits**, filter `kube_namespace:support-tickets` and `kube_deployment:support-tickets-backend`. Compare `kubernetes.memory.usage` with `kubernetes.memory.limits` immediately before each restart.
3. In **Container Restarts**, confirm `kubernetes.containers.restarts` increases for the same Deployment.

4. Open **Monitors > Manage Monitors** and inspect **Memory saturation approaching container limit** and **Pod restarts detected**. Expand the `support-tickets-backend` group.
5. Open **Infrastructure > Kubernetes > Pods**, filter `kube_namespace:support-tickets` `kube_deployment:support-tickets-backend`, and inspect Status, Restarts, Memory Usage, and Memory Limit.
6. Open **Events > Explorer** with `kube_namespace:support-tickets` and look for BackOff or container termination events. Datadog may show the restart symptom before the exact termination reason.

Record the memory value, configured limit, restart timestamp, pod name, and monitor transition. Confirm the exact `OOMKilled` reason later with `kubectl describe`.

Troubleshooting

Find the restarting pod and compare live usage with its resource specification.

```
kubectl get pods -n support-tickets
kubectl top pods -n support-tickets
kubectl describe pod <pod-name> -n support-tickets
kubectl get events -n support-tickets --sort-by=.lastTimestamp
kubectl logs <pod-name> -n support-tickets --previous
kubectl get deployment support-tickets-backend -n support-tickets -o yaml
```

Important Clues

Look for `Reason: OOMKilled`. Compare observed memory consumption with `resources.requests.memory` and `resources.limits.memory`.

Root Cause

Record the relationship between observed memory, the previous container state, and the configured limit. Confirm it with the instructor.

Fix

Restore the known baseline, then use observed utilization to propose a justified long-term request and limit.

```
./scripts/chaos/shrink-limits.sh support-tickets --undo
kubectl rollout status deployment/support-tickets-backend -n support-tickets
```

Do not remove the limit or choose an arbitrary large value.

Validation

Confirm pods remain Ready, restart counts stop increasing, memory stays below the limit, requests succeed, and both Datadog monitors recover.

DevOps Lesson

Resource limits must reflect measured workload behavior. Admission control can accept a value that is operationally unsafe.

STAR Method

Situation

Summarize the unstable workload.

Task

Explain how you separated application, Kubernetes, and infrastructure causes.

Action

Describe the memory, restart, previous-state, and resource comparison.

Result

State how stability and memory headroom were verified.

Natural Spoken Version

Use the matching example in [DevOps STAR Scenarios](#) after completing the lab.

Scenario 4: ALB and Kubernetes Ingress Routing Failure

Objective

Trace HTTP 404 responses across Datadog, Route 53, ALB, Ingress, Service, endpoints, and pods.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/break-ingress.sh food-delivery
```

The script saves the current Ingress backend and changes the live route. Allow one or two minutes for the AWS Load Balancer Controller to reconcile before testing the public URL.

Situation

Users receive HTTP 404 from the food delivery home page even though the Kubernetes workloads report healthy.

Symptoms

- The public hostname returns HTTP 404.
- Kubernetes workloads remain Running and Ready.

What You Know

- `food-delivery.<lab-domain>` resolves and the ALB accepts connections.
- Pods are Running and Ready.
- Datadog Trace Explorer shows requests to `service:food-delivery-backend` with `http.status_code:404` and resource `GET /`.

Start With Datadog

1. Open **Dashboards > SRE Lab Scenario Signals** and set the time range to the last 15 minutes.
2. Inspect **Backend GET / HTTP 404 Responses** for `service:food-delivery-backend`. This widget uses `trace.express.request.hits.by_http_status` filtered by `http.status_code:404` and metric-normalized `resource_name:get_ /`.

3. Open **Monitors > Manage Monitors > [SRE Lab] Unexpected backend root traffic** and expand the `food-delivery-backend` group.
4. Open **APM > Trace Explorer** and search `service:food-delivery-backend env:lab @http.status_code:404 resource_name:"GET /"`. Set the time range to the last 15 minutes.
5. Open a matching trace. In the trace header verify service `food-delivery-backend`, resource `GET /`, and HTTP status 404. In the Infrastructure tab record the pod and Kubernetes tags.
6. Open **Infrastructure > Kubernetes > Pods**, filter `kube_namespace:food-delivery`, and confirm frontend and backend pods remain Running and Ready.

This repository does not install the Datadog AWS integration, so ALB listeners, rules, and target health are not available in Datadog. Record the 404 trace timestamp and backend pod, then continue with AWS CLI and Kubernetes.

Troubleshooting

Trace the Kubernetes portion of the request path.

```
kubectl get ingress -A
kubectl describe ingress food-delivery -n food-delivery
kubectl get svc -n food-delivery
kubectl describe svc food-delivery-frontend -n food-delivery
kubectl get endpoints -n food-delivery
kubectl get pods -n food-delivery
```

Then trace the AWS portion. Obtain ARNs from each preceding command.

```
aws elbv2 describe-load-balancers --region us-east-1
aws elbv2 describe-listeners --load-balancer-arn <alb-arn> --region us-east-1
aws elbv2 describe-rules --listener-arn <listener-arn> --region us-east-1
aws elbv2 describe-target-health --target-group-arn <target-group-arn> --region us-east-1
aws route53 list-resource-record-sets --hosted-zone-id <hosted-zone-id>
```

Important Clues

Follow DNS to ALB, listener, target group, Ingress, Service, endpoints, pod, and application. Determine why the backend answers a request intended for the frontend.

Root Cause

Record which routing hop sends traffic to the wrong destination. Confirm it with the instructor after the exercise.

Fix

```
./scripts/chaos/break-ingress.sh food-delivery --undo  
kubectl describe ingress food-delivery -n food-delivery
```

The source-of-truth manifest is `ingress/food-delivery-ingress.yaml`, whose backend is `food-delivery-frontend:80`.

Validation

Test `https://food-delivery.<lab-domain>/`, confirm the controller reconciles the Ingress, and verify the backend `GET /` trace rate returns to zero in Datadog.

DevOps Lesson

Healthy pods do not guarantee healthy user traffic. Validate the whole routing path.

STAR Method

Situation

Summarize the user-visible 404 with healthy workloads.

Task

Explain the need to locate the failing routing hop.

Action

Describe the Datadog, AWS, and Kubernetes traffic-path checks.

Result

State how external access and trace recovery were verified.

Natural Spoken Version

Use the matching example in [DevOps STAR Scenarios](#) after completing the lab.

Scenario 5: High Application Latency Using Datadog

Objective

Use Datadog APM to identify a slow workload before investigating Kubernetes and applying a focused recovery.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/inject-latency.sh ecommerce 3000
```

The request reaches one backend pod because chaos state is stored per process. Generate application traffic and wait for APM ingestion before troubleshooting. Repeat through the documented port-forward method only when you want every backend pod affected.

Situation

Users report that ecommerce pages load slowly. Do not restart pods before scoping the problem.

Symptoms

- Ecommerce responses are intermittently slow.
- Pods remain Ready.

What You Know

- Datadog reports elevated p95 latency.
- The issue affects `ecommerce-backend` while pods remain Ready.
- The start time does not match an AWS or database change.

Start With Datadog

1. Open **Dashboards > ecommerce** and set the time range to the last 15 minutes. In **p95 Latency**, look for `p95:trace.express.request{env:lab,service:ecommerce-backend}` rising above the normal baseline.
2. Open **Dashboards > SRE Lab Scenario Signals**. Compare **Backend p95 Latency** with CPU, memory, restarts, and available replicas. The scenario should raise latency without requiring resource saturation or unhealthy pods.

3. Open **Monitors > Manage Monitors > [SRE Lab] High p95 latency** and expand the `ecommerce-backend` group. Record warning/alert start time and value.
4. Open **APM > Trace Explorer** and search `service:ecommerce-backend env:lab`. Sort or filter for the slowest traces, for example `duration:>2s`.
5. Open a slow trace and use the Flame Graph or Waterfall. Record total duration, resource name, backend pod/container tags, and where the long duration appears.
6. Open **Logs > Explorer**, set the same time range, and search `service:ecommerce-backend`. Correlate logs by timestamp and trace ID when available.
7. Open **Infrastructure > Kubernetes > Pods**, filter `kube_namespace:ecommerce kube_deployment:ecommerce-backend`, and compare CPU, memory, Ready status, and restart count.

The key scope is slow APM traces for one service while Kubernetes health and resource signals remain near baseline.

Troubleshooting

Use the Datadog service and time window to scope Kubernetes checks.

```
kubectl get pods -n ecommerce
kubectl top pods -n ecommerce
kubectl get deployment -n ecommerce
kubectl describe pod <pod-name> -n ecommerce
kubectl logs <pod-name> -n ecommerce
kubectl get events -n ecommerce --sort-by=.lastTimestamp
```

Compare slow traces with CPU, memory, replica count, events, and logs. Inspect workload behavior instead of changing unrelated infrastructure.

Important Clues

Determine why one or more backend containers add time before every response. Use trace duration and the chaos-state endpoint as evidence.

```
curl -s https://ecommerce.$(cat .lab-domain)/api/chaos
```

Root Cause

Record the workload behavior that accounts for the trace duration while Kubernetes resource metrics remain normal. Confirm it with the instructor.

Fix

Clear the workload's injected latency. Chaos state is per process, so repeat until every backend pod is reset, or reset each pod through port-forwarding as documented in the README.

```
./scripts/chaos/reset.sh ecommerce
```

Validation

Generate normal requests. Confirm p95 latency and traces return to baseline, pods remain healthy, Kubernetes metrics remain normal, and the Datadog monitor recovers.

DevOps Lesson

Use traces to isolate the slow workload before taking action. A restart may hide evidence without explaining the cause.

STAR Method

Situation

Summarize the latency report and initial scope.

Task

Explain the need to identify the affected workload before acting.

Action

Describe the dashboard, trace, Kubernetes, log, and workload-state evidence.

Result

State how normal latency and healthy pods were verified.

Natural Spoken Version

Use the matching example in [DevOps STAR Scenarios](#) after completing the lab.